

Kingsgate / Semar

How to Add a New 3rd Party Gateway

Step-by-step guide with Razorpay (dps-be, dps-fe, dps-payment-page)

Index

1. Overview of What Gets Added
 2. Backend (dps-be) – Step by Step
 3. Frontend Dashboard (dps-fe) – Step by Step
 4. Payment Page (dps-payment-page)
 5. Checklist Summary
 6. Methods to Add or Update (Quick Reference)
-

1. Overview of What Gets Added

This document lists every file and change required to add a new payment gateway (e.g. Razorpay). Follow the same pattern for any new gateway (e.g. Stripe or NewPG).

Layer	What to add / update
BE – Enum	GatewayName in <code>dps-be/src/utils/enum/enum.ts</code>
BE – Entity	New entity file (e.g. <code>razorpay.entity.ts</code>) with columns for credentials (live + sandbox)
BE – DTOs	Create + Update DTOs for the gateway (for API validation)

BE – Gateway module	Gateway module: register entity, add create/get/update in service + controller, seed data, secret keys list
BE – Channel data	Add rows in <code>gateway/data/channel.data.ts</code> for the new gateway (INCOMING/OUTGOING × UPI/BANKING/E_WALLET)
BE – Payment -system	New gateway folder (e.g. <code>razorpay/</code>) with service + module; wire into <code>PaymentSystemUtilService</code> and <code>PaymentSystemService</code> (payin + payout if needed)
BE – Payment DTOs	Add gateway name to <code>paymentMethod</code> / <code>paymentGateway</code> enums in <code>create-payment</code> and <code>assign-gateway</code> DTOs
FE – API	<code>dps-fe/src/api/gateway.ts</code> : add <code>GatewayName</code> , <code>getXxxGateway</code> , <code>updateXxx</code>
FE – Gateway tab	New key form component (e.g. <code>RazorPayKeys.tsx</code>), wire in <code>KeyForm/index.tsx</code> and <code>Gateway.tsx</code> ; add card in <code>Gateways/index.tsx</code>
Payment page	<code>dps-payment-page</code> : add to <code>PAYMENT_METHODS</code> and payment method types if gateway is selectable on checkout

2. Backend (dps-be) – Step by Step

Step 2.1 – Add gateway name to enum

File: `src/utils/enum/enum.ts`

Add the new value to `GatewayName`:

```
export enum GatewayName {
  RAZORPAY = 'RAZORPAY',
  PHONEPE = 'PHONEPE',
  // ... existing
  NEW_GATEWAY = 'NEW_GATEWAY', // add this
}
```

Step 2.2 – Create entity

File: `src/gateway/entities/razorpay.entity.ts` (create new)

Define columns: typically `incoming`, `outgoing` (booleans), and credential fields for **live** and **sandbox**. Store secrets (keys, secrets, account numbers) as strings; they will be encrypted by the gateway service before save.

```
import { Column, Entity, PrimaryGeneratedColumn } from 'typeorm';
```

```
@Entity()
export class Razorpay {
  @PrimaryGeneratedColumn()
  id: number;
```

```
  @Column()
  incoming: boolean;
```

```
  @Column()
  outgoing: boolean;
```

```
  @Column()
  key_secret: string;
```

```
  @Column()
  key_id: string;
```

```
  @Column()
  account_number: string;
```

```

@Column()
sandbox_key_id: string;

@Column()
sandbox_key_secret: string;

@Column()
sandbox_account_number: string;

}

```

For a different gateway, create e.g. `newgateway.entity.ts` with that gateway's credential fields.

Step 2.3 – Create DTOs

File: `src/gateway/dto/create-razorpay.dto.ts` (create new)

Create DTO for validation on create/update. Use `class-validator` (`@IsNotEmpty()`, `@IsString()`, `@IsBoolean()`). Update DTO uses `PartialType(CreateRazorpayDto)` so all fields are optional.

```

import { PartialType } from '@nestjs/mapped-types';
import { IsBoolean, IsNotEmpty, IsString } from 'class-validator';

export class CreateRazorpayDto {
  @IsNotEmpty()
  @IsBoolean()
  incoming: boolean;

  @IsNotEmpty()
  @IsBoolean()
  outgoing: boolean;

  @IsNotEmpty()
  @IsString()
  key_secret: string;
  // ... same for key_id, account_number, sandbox_key_id, sandbox_key_secret, sandbox_account_number
}

export class UpdateRazorpayDto extends PartialType(CreateRazorpayDto) {}

```

Note: GatewayResponseDto in gateway-response.dto.ts is shared; it only exposes id, incoming, outgoing. Secret fields are excluded. No change needed unless you want to expose more non-sensitive fields.

Step 2.4 – Gateway module: register entity, service, controller

File: src/gateway/gateway.module.ts

- Add entity to TypeOrmModule.forFeature([..., Razorpay]).

File: src/gateway/gateway.service.ts

- Inject repository: @InjectRepository(Razorpay) private readonly razorpayRepository: Repository<Razorpay>.
- Define secretTextKeysRazorpay = ['key_secret', 'key_id', 'account_number', 'sandbox_key_id', 'sandbox_key_secret', 'sandbox_account_number'] (all fields that must be encrypted).
- Implement createRazorPay(): check if record exists (throw ConflictException if yes), get seed data from loadRazorpayData(), encrypt each key in secretTextKeysRazorpay with this.jwtService.encryptValue(...), then razorpayRepository.save(...). Use a **single** forEach over secretTextKeys to encrypt (avoid double loop).
- Implement getRazorpay(): razorpayRepository.find(), return first row mapped through plainToInstance(GatewayResponseDto, ...) so secrets are excluded.
- Implement updateRazorpay(updateRazorpayDto): load existing row, merge DTO, encrypt only keys present in DTO using secretTextKeysRazorpay, then razorpayRepository.update(id, updatedData).

File: src/gateway/gateway.controller.ts

- Add endpoints: POST gateway/razorpay/create, GET gateway/razorpay, POST gateway/razorpay/update (body: UpdateRazorpayDto). Use @Roles(Role.SUB_ADMIN, Role.SUPER_ADMIN) and RolesGuard.

Step 2.5 – Seed data for create

File: `src/gateway/data/gateway.data.ts`

Add a loader that returns default values for the new gateway (used on first “create”):

```
export const loadRazorpayData = () => ({
  incoming: true,
  outgoing: true,
  key_secret: 'dummy_secret_key',
  key_id: 'dummy_secret_key',
  sandbox_key_id: 'dummy_secret_key',
  sandbox_key_secret: 'dummy_secret_key',
  account_number: '1234 1234 1234 1234',
  sandbox_account_number: '1234 1234 1234',
});
```

Step 2.6 – Channel settings seed data

File: `src/gateway/data/channel.data.ts`

Add entries for the new gateway: for each of `PaymentType.INCOMING` and `PaymentType.OUTGOING`, and each of `ChannelName.UPI`, `ChannelName.BANKING`, `ChannelName.E_WALLET`, push an object with `gatewayName: GatewayName.RAZORPAY`, `type`, `channelName`, `enabled`, `minAmount`, `maxAmount`, `upstreamFee`. (Razorpay example: INCOMING UPI/BANKING/E_WALLET enabled; OUTGOING can be disabled initially.)

These are used when “create channel settings” is run (admin) to seed `channel_settings` table. If channel settings are created once per system, add the new gateway rows to the array returned by `loadChannelData()`.

Step 2.7 – Payment-system: gateway-specific service and module

Create folder: `src/payment-system/razorpay/`

File: `razorpay.service.ts`

- Inject: gateway entity repository (Razorpay), EndUser, Payin, Identity (if needed for payout), JwtService, HttpService or SDK client.
- **getCredentials(environment)**: load single Razorpay row, pick live vs sandbox keys/account, decrypt with `jwtService.decryptValue`, return `{ key_id, key_secret, account_number }`.
- **getPayPage(getPayPageDto)**: build payment link/URL for payin (amount, customer, callback_url, etc.). Call gateway API (e.g. Razorpay payment link API). Return `{ url, trackingId, details }`. On failure, optionally update payin `gatewayError`.
- **getPaymentStatus(paymentLinkId, environment)**: call gateway API to fetch payment status; return normalized `{ status: 'SUCCESS' | 'FAILED', details }`.
- If the gateway supports **payout**: implement `makePayoutPaymentForEndUsers` and/or `makePayoutPaymentForInternalUsers` (same signature as `RazorpayService`), return `{ gatewayName, transactionId, transactionReceipt, paymentStatus, transactionDetails }`.

File: `razorpay.module.ts`

- Import `TypeOrmModule.forFeature([Razorpay, EndUser, Payin, Identity, ...])`, `JwtModule`, `HttpModule` if needed.
- Providers: `RazorpayService`; exports: `RazorpayService`.

Step 2.8 – Wire gateway into PaymentSystemModule and Util

File: `src/payment-system/payment-system.module.ts`

- Import `RazorpayModule`.
- Add `Razorpay` to `TypeOrmModule.forFeature([...])`.
- Add `RazorpayService` to providers (if not provided by `RazorpayModule`).

File: `src/payment-system/payment-system.util.service.ts`

- Inject `Razorpay` repository and `RazorpayService`.
- **getGatewayForPayin / getFirstEnabledGateway**: in the `switch (gateway)`, add case `GatewayName.RAZORPAY: isGatewayEnabled = await this.razorpayRepository.findOne({ where: whereConditions }); break;`

- **assignPaymentMethodForPayinOrder:** when computing `selectedPaymentMode` (e.g. from merchant config or round-robin), add branch for `GatewayName.RAZORPAY`; call `this.getPayPage({ ..., gateway: GatewayName.RAZORPAY, ... })` and then update `payin` with `trackingId`, `gatewayPaymentLink`.
- **getPayPage(getPayPageDto):** add `if (gateway === GatewayName.RAZORPAY)` return `await this.razorpayService.getPayPage(getPayPageDto);`
- If you use **proportional or ratio-based** gateway selection: add `'razorpay'` to the `ratios` list and handle `GatewayName.RAZORPAY` in the same way as existing gateways (e.g. in the loop that assigns by diff).

File: `src/payment-system/payment-system.service.ts`

- Inject `RazorpayService`.
- **Payin status polling:** where `payin` order's `gatewayName` is checked (e.g. `if (payinOrder.gatewayName === GatewayName.RAZORPAY)`), call `razorpayService.getPaymentStatus(...)` and map result to order status.
- **Payout (if applicable):** in the payout flow where gateway is selected, add `case GatewayName.RAZORPAY` and call `razorpayService.makePayoutPaymentForEndUsers` or `makePayoutPaymentForInternalUsers`.

Step 2.9 – Payment DTOs (`paymentMethod` / `paymentGateway`)

File: `src/payment-system/dto/createPaymentOrder.dto.ts`

Where `paymentMethod` is validated with `@IsEnum([...])`, add `'razorpay'` to the allowed list.

File: `src/payment-system/dto/getPayPage.dto.ts`

Same: add `'razorpay'` to the enum array for `paymentMethod` / `paymentGateway` if used in `assign-gateway` or `get-pay-page`.

3. Frontend Dashboard (dps-fe) – Step by Step

Step 3.1 – API: gateway name and functions

File: `src/api/gateway.ts`

- Add `RAZORPAY = "RAZORPAY"` to `GatewayName` enum (if not already there).
- Add `getRazorpayGateway = async () =>`
`axiosInstance.get('/gateway/razorpay', { headers: { Authorization: `Bearer ${getAuthToken()}` } }); return response.data.`
- Add `updateRazorpay = async (type?, value?, additionalData?) =>`
`axiosInstance.post('/gateway/razorpay/update', payload, { headers: { Authorization: ... } }).` Payload: if `type === null`, send `additionalData` (keys/credentials); else send `{ [type.toLowerCase()]: value, ...additionalData }`.

Step 3.2 – Key form component (copy from existing gateway)

File:

`src/pages/Dashboard/pages/Admin/Gateways/KeyForm/Components/RazorPayKeys.tsx`
(create new)

Copy from e.g. `PhonepeKeys.tsx` or `CashfreeKeys.tsx`. Props: `fields`, `errors`, `setFields`, `setErrors`, `currentTab`.

- For `currentTab === 'live'`: render inputs for live credentials (e.g. `key_id`, `key_secret`, `account_number`).
- For `currentTab === 'sandbox'`: render inputs for sandbox credentials (e.g. `sandbox_key_id`, `sandbox_key_secret`, `sandbox_account_number`).
- Use `TextInput` (Mantine), `value={fields.key_name}`, `onChange={handleChange('key_name')}`, `error={errors.key_name}`.

Step 3.3 – KeyForm index: state, validation, save, tab panel

File: `src/pages/Dashboard/pages/Admin/Gateways/KeyForm/index.tsx`

- Add Razorpay fields to `liveFields` and `sandboxFields` state (e.g. `key_id`, `key_secret`, `account_number`, `sandbox_key_id`, `sandbox_key_secret`, `sandbox_account_number`).
- Add corresponding entries to `errors` state.

- In `validateFields()`: for `gateway === GatewayName.RAZORPAY`, validate live and sandbox tabs (required fields); set `newErrors` and `isValid`.
- In `handleSave()`: for `gateway === GatewayName.RAZORPAY`, call `updateRazorpay(null, null, { key_id, key_secret, account_number })` for live and `updateRazorpay(null, null, { sandbox_key_id, sandbox_key_secret, sandbox_account_number })` for sandbox.
- In `getTabPanelComponent()`: add `else if (gateway === GatewayName.RAZORPAY)` return `<RazorPayKeys ... />` with `fields`, `errors`, `setFields`, `setErrors`, `currentTab`.
- In `getGatewayLogo()`: add `if (gateway === GatewayName.RAZORPAY)` return `RazorpayIcon`; (and add Razorpay icon image under `assets/` if needed).

Step 3.4 – Gateway page: fetch and toggles

File: `src/pages/Dashboard/pages/Admin/Gateways/Gateway.tsx`

- In `useEffect fetch`: add `if (gateway === GatewayName.RAZORPAY)` `response = await getRazorpayGateway();`
- After `fetch`: `if (gateway === GatewayName.RAZORPAY)` `setEnabledForIncoming(response.incoming);` `setEnabledForOutgoing(response.outgoing);`
- In `updateGateway`: add `if (gateway === GatewayName.RAZORPAY)` `await updateRazorpay(type, value);` and update local state for incoming/outgoing.
- In `getGatewayLogo`: add `if (gateway === GatewayName.RAZORPAY)` return `RazorpayIcon`;

Step 3.5 – Gateways list: show new gateway card

File: `src/pages/Dashboard/pages/Admin/Gateways/index.tsx`

Add a new `Grid.Col` with `<Gateway gateway={GatewayName.RAZORPAY} />`. Optionally update `getAllGateways` in API to return `["RAZORPAY", "PHONEPE", ...]` if you drive the list from BE.

4. Payment Page (dps-payment-page)

If the checkout flow lets the user **choose** a gateway (e.g. “Pay with Razorpay”):

- **File:** `src/utils/constants.ts` – Add `razorpay: "razorpay"` to `PAYMENT_METHODS`.
- **File:** `src/types/payment.ts` – Extend `paymentMethod` type to include `"razorpay"` where applicable.
- Where payment method is sent to BE (create-payment-order or assign-payment-gateway), the value `"razorpay"` must be allowed; BE already accepts it once DTOs are updated as in Step 2.9.

If the gateway is only assigned by the backend (no dropdown on payment page), no change is required in `dps-payment-page`.

5. Checklist Summary

#	Location	Action
1	BE enum	Add <code>GatewayName.RAZORPAY</code>
2	BE gateway/entities	Create <code>razorpay.entity.ts</code>
3	BE gateway/dto	Create <code>create-razorpay.dto.ts</code> (Create + Update)
4	BE gateway/data	Add <code>loadRazorpayData()</code> in <code>gateway.data.ts</code>
5	BE gateway/data	Add Razorpay rows in <code>channel.data.ts</code>

6	BE gateway.module	Register Razorpay entity
7	BE gateway.service	Inject repo; add secretTextKeys; createRazorPay, getRazorpay, updateRazorpay
8	BE gateway.controller	POST create, GET get, POST update for razorpay
9	BE payment-system/razorpay	razorpay.service.ts (getCredentials, getPayPage, getPaymentStatus, payout if needed)
10	BE payment-system/razorpay	razorpay.module.ts
11	BE payment-system.module	Import RazorpayModule; TypeORM Razorpay; providers RazorpayService
12	BE payment-system.util.service	Inject repo + RazorpayService; getGatewayForPayin/getFirstEnabledGateway; assignPaymentMethodForPayinOrder; getPayPage; ratio logic if any
13	BE payment-system.service	Inject RazorpayService; payin status; payout case

14	BE payment-system DTOs	Add 'razorpay' to paymentMethod/paymentGateway enums
15	FE api/gateway.ts	GatewayName.RAZORPAY; getRazorpayGateway; updateRazorpay
16	FE KeyForm/Components	RazorPayKeys.tsx (live + sandbox inputs)
17	FE KeyForm/index.tsx	State, validate, handleSave, getTabPanelComponent, getGatewayLogo for RAZORPAY
18	FE Gateway.tsx	Fetch getRazorpayGateway; updateRazorpay; getGatewayLogo
19	FE Gateways/index.tsx	Add <Gateway gateway={GatewayName.RAZORPAY} />
20	dps-payment-page	PAYMENT_METHODS + types if gateway is user-selectable

6. Methods to Add or Update (Quick Reference)

Layer	Method / endpoint	Purpose
BE GatewayService	createRazorPay()	Seed first row; encrypt secrets
BE GatewayService	getRazorpay()	Return config (no secrets)
BE GatewayService	updateRazorpay(dto)	Update row; encrypt provided secrets
BE GatewayController	POST gateway/razorpay/create	Create config
BE GatewayController	GET gateway/razorpay	Get config
BE GatewayController	POST gateway/razorpay/update	Update config
BE RazorpayService	getCredentials(environment)	Decrypt and return live/sandbox keys
BE RazorpayService	getPayPage(dto)	Create payment link for payin
BE RazorpayService	getPaymentStatus(paymentLinkId, environment)	Fetch status from gateway

BE RazorpayService	makePayoutPaymentForEndUsers / makePayoutPaymentForInternalUsers	If payout supported
BE PaymentSystemUtilService	getGatewayForPayin, getFirstEnabledGateway	Add case GatewayName.RAZORPAY
BE PaymentSystemUtilService	assignPaymentMethodForPayin Order, getPayPage	Call razorpayService when gateway is RAZORPAY
BE PaymentSystemService	Payin status check, payout flow	Call razorpayService.getPayment Status / makePayout*
FE api/gateway	getRazorpayGateway(), updateRazorpay(...)	GET config; POST update
FE KeyForm	RazorPayKeys component; validate; handleSave	Form for keys; call updateRazorpay
FE Gateway.tsx	fetch + updateGateway + getGatewayLogo	Load config; toggle incoming/outgoing; show logo

End of document. Use Razorpay as the reference implementation; for a new gateway, duplicate and rename (entity, DTOs, service, controller, FE form, API, and enum) following this list.